Instituto Politécnico Nacional

Escuela Superior de Cómputo

Ingeniería en Sistemas Computacionales



Práctica 03: Codificación Voraz de Huffman

Alumno: Linares Ojeda Carlos Elías

No. de boleta: 2024630337

Grupo: 3CM4

Unidad de aprendizaje:
Análisis y Diseño de Algoritmos

Profesor: Franco Martínez Edgardo Adrián

Ciudad de México, 2024

Índice

1. Introducción **3**

2. Objetivo **3**

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Estado del arte **4**

3.1. Compresión de datos sin pérdida 4

3.2. Antecedentes: de Shannon a Huffman 4

3.3. Códigos de longitud fija y variable 5

3.4. Representación mediante árboles binarios 5

3.5. Entropía y límites de la compresión 6

3.6. Algoritmo de Huffman 7

3.7. Ejemplo paso a paso: construcción del árbol 8

3.8. Codificación y decodificación del ejemplo 9

3.9. Optimalidad de la elección voraz 11

3.10. Complejidad del algoritmo 12

3.11. Variantes y aplicaciones actuales 12

3.12. Relación con la práctica 14

4. Desarrollo **14**

4.1. Planteamiento del problema 14

4.2. Metodología y entorno de pruebas 15

4.2.1. Entorno experimental 15

4.2.2. Pruebas funcionales 15

4.3. Estructuras de datos 15

4.4. Implementación del codificador 16

4.5. Formato de almacenamiento 17
4.6. Implementación del decodificador 17
4.7. Ejecución y automatización 18
4.8. Medición de tiempo y compresión 18
4.9. Análisis de complejidad 19

5. Gráficos y diagramas esquemáticos **20**
5.1. Flujo de los programas 20
5.2. Árbol del ejemplo 20

6. Resultados **21**
6.1. Compresión obtenida 21
6.2. Tiempos de ejecución y ajuste 22
6.3. Pruebas funcionales 24
6.4. Limitaciones 25

7. Cuestionario **25**

8. Conclusiones **27**

Referencias bibliográficas **28**

1. Introducción

La compresión de datos permite reducir el espacio de almacenamiento y la cantidad de información que debe transmitirse. En una compresión sin pérdida, el contenido recuperado debe coincidir byte por byte con la entrada; disminuir el tamaño carece de utilidad si no se conserva esa propiedad.

La práctica estudia una implementación en C de la codificación de Huffman sobre un alfabeto de bytes. A partir de las frecuencias observadas se construye un árbol binario y se asignan códigos de longitud variable: los bytes frecuentes reciben códigos cortos y los poco frecuentes, códigos largos. El método de Huffman produce un código prefijo de redundancia mínima [1] y es un ejemplo clásico de algoritmo voraz, en el que una secuencia de decisiones localmente óptimas conduce a una solución global óptima [2]. Además del algoritmo, se analizan los costos del formato concreto: cabecera, tabla de frecuencias, empaquetado de bits y recuperación.

El documento presenta los objetivos, el estado del arte de la codificación de Huffman —incluida la construcción del árbol paso a paso—, la implementación, el análisis de complejidad, diagramas, los resultados experimentales para 21 archivos de distintos tamaños y formatos, las pruebas funcionales, el cuestionario y las conclusiones.

2. Objetivo

2.1. Objetivo general

Implementar y analizar la codificación voraz de Huffman y su decodificación para archivos binarios, considerando recuperación exacta, espacio ocupado, tiempo de procesamiento y complejidad computacional.

2.2. Objetivos específicos

- Describir el funcionamiento del algoritmo de Huffman: construcción del árbol, asignación de códigos, codificación y decodificación.
- Contar las frecuencias de los valores de byte presentes en un archivo.
- Construir el árbol mediante fusiones sucesivas de los dos árboles de menor frecuencia y generar los códigos correspondientes.

- Almacenar los códigos en bits y conservar los metadatos necesarios para reconstruir la entrada.
- Medir el tamaño resultante y los tiempos de codificación y decodificación para archivos de distintos tamaños y formatos, incluyendo los costos auxiliares.
- Distinguir la complejidad respecto al tamaño del archivo de la complejidad respecto al número de símbolos.
- Verificar la recuperación exacta comparando el contenido original con el decodificado.

3. Estado del arte

3.1. Compresión de datos sin pérdida

Un compresor sin pérdida transforma una secuencia de símbolos en una representación más corta, de la cual puede recuperarse exactamente la secuencia original. Es indispensable cuando cualquier diferencia invalida el resultado, como en texto, código fuente, programas ejecutables o registros de datos. Los métodos con pérdida, empleados en audio, imagen y video, descartan información poco perceptible a cambio de una reducción mayor.

Un sistema de compresión sin pérdida se divide habitualmente en dos etapas [3]: un *modelo*, que estima qué símbolos son probables, y un *codificador*, que asigna a cada símbolo una representación binaria acorde con esa estimación. Los métodos de diccionario, como LZ77 [4], sustituyen secuencias repetidas por referencias a apariciones anteriores; los métodos estadísticos, también llamados de entropía, aprovechan que algunos símbolos aparecen con mayor frecuencia que otros. La codificación de Huffman pertenece a este segundo grupo. Los compresores de uso general suelen combinar ambos enfoques: primero eliminan repeticiones y después codifican los símbolos resultantes con un código de entropía, como hace el formato DEFLATE [5].

3.2. Antecedentes: de Shannon a Huffman

En 1948, Claude E. Shannon formalizó la teoría de la información y definió la entropía de una fuente, que establece el número promedio mínimo de bits por símbolo con el que puede representarse sin pérdida [6]. Poco después, Robert M. Fano describió un método de construcción descendente [7], conocido como codificación de Shannon-Fano: los símbolos

se ordenan por probabilidad y se dividen recursivamente en dos grupos de probabilidad total semejante, asignando 0 a un grupo y 1 al otro. El método produce códigos prefijo eficientes, pero no garantiza la longitud media mínima.

En 1951, David A. Huffman, alumno del curso de teoría de la información de Fano en el Instituto Tecnológico de Massachusetts (MIT), abordó como trabajo final el problema de encontrar el código binario más eficiente. Su solución, publicada en 1952 [1], invierte el sentido de la construcción: en lugar de dividir el conjunto de símbolos desde la raíz, parte de las hojas y fusiona repetidamente los dos símbolos menos probables. Este procedimiento ascendente obtiene la longitud media mínima entre todos los códigos prefijo para las probabilidades dadas, y sigue siendo la base de numerosos formatos de compresión.

3.3. Códigos de longitud fija y variable

Un código binario asigna a cada símbolo de un alfabeto Σ una palabra de código formada por ceros y unos. En un código de longitud fija todas las palabras tienen $\lceil \log_2 |\Sigma| \rceil$ bits; por ejemplo, la representación habitual de un archivo usa ocho bits por byte. Esta representación es simple, pero dedica el mismo espacio a un símbolo que aparece miles de veces que a uno que aparece una sola vez.

Un código de longitud variable puede asignar menos bits a los símbolos frecuentes. Sin embargo, al concatenar palabras de distinta longitud debe ser posible separarlas sin ambigüedad. Con el código $a = \texttt{0}$, $b = \texttt{01}$ y $c = \texttt{10}$, la cadena `010` admite dos lecturas, $a\,c$ y $b\,a$, por lo que el código no es de decodificación única.

Un código es *prefijo*, o libre de prefijos, si ninguna palabra de código es prefijo de otra. En ese caso la decodificación es instantánea: al leer los bits de izquierda a derecha, en cuanto se completa una palabra se sabe que terminó, sin consultar bits posteriores. El código $a = \texttt{0}$, $b = \texttt{10}$ y $c = \texttt{11}$ es prefijo, y la cadena `01011` sólo admite la lectura $a\,b\,c$. Además, cualquier código de decodificación única puede sustituirse por un código prefijo con las mismas longitudes [8, 9], de modo que basta buscar el código óptimo entre los códigos prefijo [10].

3.4. Representación mediante árboles binarios

Un código prefijo binario se representa con un árbol binario cuyas hojas son los símbolos. La palabra de código de un símbolo es el camino desde la raíz hasta su hoja: cada arista hacia un hijo izquierdo aporta un 0 y cada arista hacia un hijo derecho, un 1. Como los símbolos sólo ocupan hojas, ningún camino completo es prefijo de otro. La longitud del código de un símbolo c coincide con la profundidad $d_T(c)$ de su hoja en el árbol T.

Si $f(c)$ es la frecuencia del símbolo c en el mensaje, el número de bits necesario para codificarlo, llamado *costo* del árbol [2], es

$$B(T) = \sum_{c \in \Sigma} f(c)\, d_T(c). \tag{1}$$

Un árbol óptimo es siempre *completo*: todo nodo interno tiene dos hijos. Si un nodo interno tuviera un solo hijo, podría eliminarse y subir su subárbol un nivel; todos los códigos de ese subárbol se acortarían en un bit sin perder la propiedad de prefijo. Un árbol completo con k hojas tiene $k-1$ nodos internos y $2k-1$ nodos en total.

3.5. Entropía y límites de la compresión

Si el símbolo a_i aparece con probabilidad p_i, la entropía de la fuente es

$$H = -\sum_{i=1}^{k} p_i \log_2 p_i \qquad \text{bits por símbolo.} \tag{2}$$

Las longitudes ℓ_i de cualquier código de decodificación única satisfacen la desigualdad de Kraft-McMillan [8, 9]

$$\sum_{i=1}^{k} 2^{-\ell_i} \leq 1, \tag{3}$$

y, recíprocamente, para longitudes que la cumplen existe un código prefijo con esas longitudes. De estas propiedades se deduce que la longitud media $\overline{\ell} = \sum_i p_i \ell_i$ de cualquier código de decodificación única cumple $\overline{\ell} \geq H$, y que el código de Huffman satisface [10]

$$H \leq \overline{\ell}_{\text{Huffman}} < H + 1. \tag{4}$$

La igualdad $\overline{\ell} = H$ se alcanza sólo cuando todas las probabilidades son potencias de $1/2$. Como cada símbolo recibe al menos un bit, la diferencia respecto a la entropía puede ser considerable cuando un símbolo es muy probable.

En esta práctica los símbolos son bytes, de modo que $0 \leq H \leq 8$ bits por byte. Un archivo cuyos bytes son casi equiprobables tiene una entropía cercana a ocho bits y no puede reducirse con un código por símbolos. Es el caso de los archivos que ya pasaron por un compresor, como ZIP, RAR, 7z, MP3, JPEG o video codificado, cuya redundancia estadística ya fue eliminada en gran medida [11].

3.6. Algoritmo de Huffman

El algoritmo construye el árbol óptimo de abajo hacia arriba. Parte de un bosque con una hoja por símbolo, cuyo peso es la frecuencia del símbolo, y repite una elección voraz: retirar los dos árboles de menor peso y unirlos bajo una nueva raíz cuyo peso es la suma de ambos. Cada fusión reduce en uno el número de árboles, por lo que después de $k-1$ fusiones queda un único árbol. El algoritmo 1 expresa el procedimiento con una cola de prioridad mínima Q [2].

Algoritmo 1 Construcción del árbol de Huffman con una cola de prioridad mínima.

Entrada: alfabeto Σ con $k = |\Sigma| \geq 1$ símbolos y frecuencias $f(c) > 0$.
Salida: raíz de un árbol de Huffman para Σ.

```
1:  función HUFFMAN(Σ, f)
2:      Q ← ∅
3:      para todo c ∈ Σ hacer
4:          crear una hoja z con símbolo c y peso f(c)
5:          INSERTAR(Q, z)
6:      fin para
7:      para i ← 1 hasta k − 1 hacer
8:          x ← EXTRAERMIN(Q)
9:          y ← EXTRAERMIN(Q)
10:         crear un nodo z con hijo izquierdo x e hijo derecho y
11:         z.peso ← x.peso + y.peso
12:         INSERTAR(Q, z)
13:     fin para
14:     devolver EXTRAERMIN(Q)
15: fin función
```

En conjunto, la codificación de Huffman de un archivo consta de los siguientes pasos:

1. Contar la frecuencia de cada símbolo del mensaje.
2. Crear una hoja por cada símbolo con frecuencia positiva.
3. Mientras quede más de un árbol, fusionar los dos de menor peso (algoritmo 1).
4. Recorrer el árbol desde la raíz, asignando 0 a cada rama izquierda y 1 a cada rama derecha; el camino hasta cada hoja es el código de su símbolo.
5. Sustituir cada símbolo del mensaje por su código y agrupar los bits en bytes.

6. Almacenar, junto con los datos, la información necesaria para reconstruir el código —frecuencias, longitudes o el propio árbol— y para saber dónde termina el mensaje.

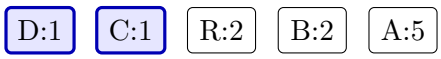
Para decodificar se reconstruye el mismo árbol y se leen los bits uno a uno: se parte de la raíz, se desciende a la izquierda con un 0 y a la derecha con un 1, y al llegar a una hoja se emite su símbolo y se regresa a la raíz.

3.7. Ejemplo paso a paso: construcción del árbol

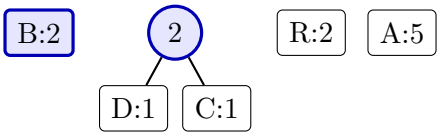
Considérese el mensaje `ABRACADABRA`, de $n = 11$ caracteres. Sus frecuencias son $f(\mathrm{A}) = 5$, $f(\mathrm{B}) = 2$, $f(\mathrm{R}) = 2$, $f(\mathrm{C}) = 1$ y $f(\mathrm{D}) = 1$; hay $k = 5$ símbolos, por lo que se requieren cuatro fusiones. La tabla 1 y la figura 1 muestran el bosque al inicio de cada paso, ordenado por peso como lo hace la implementación de esta práctica (sección 4.4); en cada paso se resaltan los dos árboles que se fusionan.

Tabla 1: Evolución del bosque durante la construcción del árbol para `ABRACADABRA`.

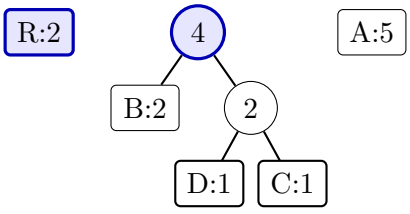
Paso	**Árboles ordenados por peso**	**Se fusionan**	**Nuevo árbol**
1	D:1, C:1, R:2, B:2, A:5	D:1 y C:1	T_1, peso 2
2	B:2, T_1:2, R:2, A:5	B:2 y T_1:2	T_2, peso 4
3	R:2, T_2:4, A:5	R:2 y T_2:4	T_3, peso 6
4	A:5, T_3:6	A:5 y T_3:6	T_4, peso 11 (raíz)



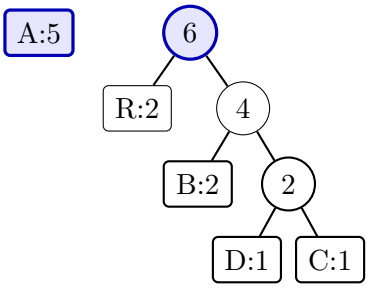
(a) Paso 1: cinco hojas.



(b) Paso 2: T_1 agrupa a D y C.



(c) Paso 3: T_2 agrupa a B y T_1.



(d) Paso 4: T_3 agrupa a R y T_2.

Figura 1: Bosque al inicio de cada paso. Los árboles resaltados son los dos de menor peso, que se fusionan en ese paso.

En el paso 1 sólo hay una elección posible: las hojas D y C, de peso uno, forman T_1.

En el paso 2 existen tres árboles de peso dos —B, R y T_1— y cualquier par entre ellos es una elección voraz válida; el ordenamiento de la implementación coloca primero a B y a T_1, que forman T_2. En el paso 3 se unen R y T_2, y en el paso 4 la hoja A con T_3. El peso de la raíz es once, igual al número de caracteres del mensaje. Al etiquetar las aristas se obtienen los códigos de la tabla 2 y el árbol de la figura 2a.

Tabla 2: Códigos obtenidos para `ABRACADABRA`.

Símbolo	f	p	**Código**	ℓ	$f\ell$
A	5	0.4545	`0`	1	5
R	2	0.1818	`10`	2	4
B	2	0.1818	`110`	3	6
D	1	0.0909	`1110`	4	4
C	1	0.0909	`1111`	4	4
Total	11	1.0000			23

El mensaje requiere $B = 23$ bits, es decir, $\overline{\ell} = 23/11 \approx 2.091$ bits por símbolo. Un código de longitud fija necesitaría $\lceil \log_2 5 \rceil = 3$ bits por símbolo (33 bits), y la representación de ocho bits, 88 bits: el código de Huffman ahorra 30.30 % y 73.86 %, respectivamente. La entropía es $H \approx 2.040$ bits por símbolo; se cumple $H \leq \overline{\ell} < H + 1$ y la redundancia es de sólo 0.051 bits por símbolo. La suma de Kraft, $2^{-1} + 2^{-2} + 2^{-3} + 2 \cdot 2^{-4} = 1$, confirma que el árbol es completo.

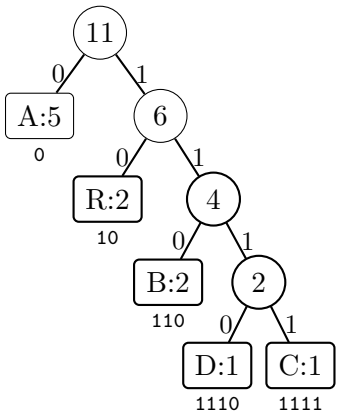
Si en el paso 2 se hubieran elegido B y R, se obtendría el árbol de la figura 2b, con A = `0` y códigos de tres bits para B, R, C y D. Su costo es $5 \cdot 1 + (2 + 2 + 1 + 1) \cdot 3 = 23$ bits, el mismo. Los códigos de Huffman no son únicos: todos alcanzan el costo mínimo, pero pueden diferir en la longitud máxima y en la dispersión de las longitudes. Por ello, cuando sólo se conservan las frecuencias, el decodificador debe repetir exactamente las decisiones de desempate del codificador.

3.8. Codificación y decodificación del ejemplo

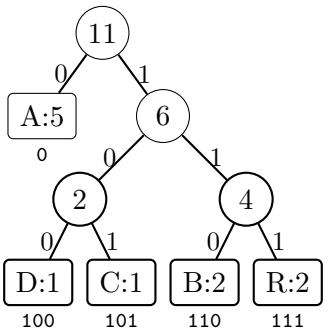
La tabla 3 sustituye cada carácter de `ABRACADABRA` por su código.

Tabla 3: Codificación de `ABRACADABRA` símbolo por símbolo.

Posición	1	2	3	4	5	6	7	8	9	10	11
Símbolo	A	B	R	A	C	A	D	A	B	R	A
Código	`0`	`110`	`10`	`0`	`1111`	`0`	`1110`	`0`	`110`	`10`	`0`



(a) Árbol de la implementación; longitudes 1, 2, 3, 4 y 4.



(b) Alternativa al fusionar B y R en el paso 2; longitudes 1, 3, 3, 3 y 3.

Figura 2: Dos árboles de Huffman para `ABRACADABRA`. Ambos codifican el mensaje con 23 bits; debajo de cada hoja aparece su código.

Al concatenar los códigos se obtienen 23 bits, que se agrupan en bytes desde el bit más significativo:

$$\underbrace{01101001}_{69_{16}} \qquad \underbrace{11101110}_{EE_{16}} \qquad \underbrace{0110100\boxed{0}}_{68_{16}}$$

El último byte contiene siete bits útiles, por lo que se completa con un bit de relleno igual a cero, marcado con un recuadro. En hexadecimal, el flujo es `69 EE 68`: tres bytes en lugar de once.

La decodificación recorre el árbol de la figura 2a con esos bits, como muestra la tabla 4. Después del undécimo símbolo sólo queda el bit de relleno. Ese bit coincide con el código de A, de modo que un decodificador que procesara todos los bits recuperaría `ABRACADABRAA`. Por ello el formato debe indicar dónde termina el mensaje, ya sea almacenando el número de símbolos —como hace esta práctica— o añadiendo un símbolo especial de fin, como el símbolo 256 de fin de bloque de DEFLATE [5].

Tabla 4: Traza de la decodificación del flujo `69 EE 68` con $n = 11$.

Paso	Bits leídos	Recorrido desde la raíz	Hoja	Emitidos
1	`0`	$11 \to$ A	A	1
2	`110`	$11 \to 6 \to 4 \to$ B	B	2
3	`10`	$11 \to 6 \to$ R	R	3
4	`0`	$11 \to$ A	A	4
5	`1111`	$11 \to 6 \to 4 \to 2 \to$ C	C	5
6	`0`	$11 \to$ A	A	6
7	`1110`	$11 \to 6 \to 4 \to 2 \to$ D	D	7
8	`0`	$11 \to$ A	A	8
9	`110`	$11 \to 6 \to 4 \to$ B	B	9
10	`10`	$11 \to 6 \to$ R	R	10
11	`0`	$11 \to$ A	A	11
—	`0` (relleno)	no se procesa: ya se emitieron n símbolos		11

3.9. Optimalidad de la elección voraz

La corrección del algoritmo se apoya en dos propiedades, demostradas formalmente en [2].

Lema 1 (Elección voraz)**.** *Sean x y y dos símbolos de menor frecuencia en Σ. Existe un árbol óptimo en el que x y y son hojas hermanas de profundidad máxima.*

Idea de la demostración. Sea T un árbol óptimo y sean a y b hojas hermanas de profundidad máxima, que existen porque T es completo. Sin pérdida de generalidad, $f(x) \le f(y)$ y $f(a) \le f(b)$; como x y y tienen las frecuencias mínimas, $f(x) \le f(a)$ y $f(y) \le f(b)$. Además, $d_T(x) \le d_T(a)$. Al intercambiar x y a se obtiene un árbol T' con

$$B(T') - B(T) = \big(f(x) - f(a)\big)\big(d_T(a) - d_T(x)\big) \le 0,$$

pues el primer factor no es positivo y el segundo no es negativo. Lo mismo ocurre al intercambiar y y b. El árbol resultante no es más costoso que T; por lo tanto, también es óptimo y tiene a x y y como hermanas en el nivel más profundo. □

Lema 2 (Subestructura óptima)**.** *Sea Σ' el alfabeto que resulta de sustituir x y y por un nuevo símbolo z con $f(z) = f(x) + f(y)$. Si T' es un árbol óptimo para Σ', el árbol T que se obtiene al reemplazar la hoja z por un nodo interno con hijos x y y es óptimo para Σ.*

Idea de la demostración. Como $d_T(x) = d_T(y) = d_{T'}(z) + 1$, de (1) se obtiene $B(T) = B(T') + f(x) + f(y)$. Si T no fuera óptimo, existiría un árbol óptimo T'' con $B(T'') < B(T)$

y, por el lema 1, podría elegirse de modo que x y y sean hermanas. Al reemplazarlas por una hoja z se obtendría un árbol para Σ' con costo $B(T'') - f(x) - f(y) < B(T')$, lo que contradice la optimalidad de T'. □

Teorema 1. *El algoritmo de Huffman produce un código prefijo óptimo.*

La demostración es por inducción sobre el número de símbolos: cada iteración realiza la elección del lema 1 y reduce el problema al alfabeto Σ' del lema 2, que tiene un símbolo menos [2, 1].

3.10. Complejidad del algoritmo

Con una cola de prioridad implementada como montículo binario, construir la cola inicial cuesta $O(k)$ y cada una de las $k - 1$ iteraciones realiza dos extracciones y una inserción de costo $O(\log k)$; el total es $O(k \log k)$ [2]. Si las frecuencias ya están ordenadas, van Leeuwen mostró que bastan dos colas simples —una con las hojas y otra con los nodos creados, que surgen en orden no decreciente de peso— para construir el árbol en tiempo $O(k)$ [12].

A estas cotas se suman el conteo de frecuencias, $O(n)$, y la codificación y decodificación del mensaje, proporcionales a n y al número de bits B escritos o leídos. La construcción del árbol depende sólo de k; con un alfabeto pequeño y fijo, como los 256 valores de un byte, su costo es despreciable frente al procesamiento de archivos grandes. En la práctica, los decodificadores eficientes sustituyen el recorrido bit a bit por tablas indexadas con varios bits a la vez [3].

3.11. Variantes y aplicaciones actuales

Desde su publicación, el método ha dado lugar a variantes que reducen el costo de almacenar el modelo, acotan la longitud de los códigos o eliminan la necesidad de dos pasadas. La tabla 5 resume las principales.

Códigos canónicos. Un código canónico conserva las longitudes de un código de Huffman, pero asigna las palabras en orden creciente de longitud y, dentro de cada longitud, en el orden de los símbolos [13]. Así, el decodificador sólo necesita la longitud de cada código, no el árbol ni las frecuencias. DEFLATE, base de ZIP, gzip y PNG, describe de esta manera sus códigos [5].

Códigos de longitud limitada. Algunos formatos acotan la longitud máxima de los códigos para simplificar la decodificación: DEFLATE admite hasta 15 bits y JPEG,

hasta 16 [5, 14]. El algoritmo *package-merge* obtiene el código óptimo bajo esa restricción [15].

Codificación adaptativa. La variante estática requiere dos pasadas —una para contar y otra para codificar— y debe almacenar el modelo. Las variantes adaptativas actualizan el árbol después de cada símbolo, de modo que el codificador y el decodificador mantienen el mismo modelo sin transmitirlo y procesan el mensaje en una sola pasada; destacan el algoritmo FGK, de Faller, Gallager y Knuth [16, 17], y el de Vitter [18].

Más allá de un número entero de bits. Un código de Huffman asigna al menos un bit a cada símbolo. La codificación aritmética representa el mensaje completo como un subintervalo de $[0, 1)$ y se aproxima a la entropía incluso con símbolos muy probables [19]. Los sistemas de numeración asimétricos (ANS) obtienen una compresión cercana a la aritmética con una velocidad comparable a la de Huffman [20].

Aplicaciones. La codificación de Huffman forma parte de estándares de uso cotidiano: DEFLATE [5], la codificación de entropía de JPEG [14] y de MP3 [21], la compresión de cabeceras de HTTP/2 mediante un código estático predefinido [22], Brotli [23] y Zstandard, que la emplea para los literales y usa ANS para el resto de la información [24]. En todos estos casos Huffman es la etapa final de un sistema cuyo modelo ya transformó los datos, por ejemplo con LZ77, y no un compresor aislado.

Tabla 5: Variantes de la codificación de Huffman y usos representativos.

Variante	**Característica principal**	**Uso representativo**
Estática	Dos pasadas: contar y codificar; el modelo se almacena con los datos.	Esta práctica.
Canónica	Las palabras se asignan en orden de longitud; basta almacenar las longitudes.	DEFLATE (ZIP, gzip, PNG).
De longitud limitada	Código óptimo con una longitud máxima prefijada (*package-merge*).	DEFLATE (15 bits), JPEG (16 bits).
Adaptativa (FGK, Vitter)	Una pasada; el árbol se actualiza tras cada símbolo y no se transmite.	Codificación en flujo sin conocer el mensaje completo.
Estática predefinida	Código fijo conocido por emisor y receptor, calculado con estadísticas típicas.	Cabeceras de HTTP/2 (HPACK).

3.12. Relación con la práctica

La implementación desarrollada corresponde a la variante estática sobre un alfabeto de 256 bytes, con un modelo de orden cero: estima la probabilidad de cada byte con su frecuencia en todo el archivo, sin considerar el contexto. Recorre el archivo una vez para contar y otra para codificar. En lugar de una cola de prioridad, ordena el arreglo de árboles con QuickSort antes de cada fusión; en lugar de almacenar el árbol o un código canónico, guarda la tabla de frecuencias en un archivo de texto, y el decodificador reconstruye el mismo árbol con idéntico procedimiento. Las secciones siguientes describen estas decisiones y su efecto en el espacio y el tiempo.

4. Desarrollo

4.1. Planteamiento del problema

La práctica solicita implementar en C la codificación voraz de Huffman y su decodificación para archivos de cualquier tipo, medir la compresión y los tiempos de ejecución para entradas desde bytes hasta gigabytes, representar el comportamiento experimental y determinar cotas de complejidad. El código considera los 256 posibles valores de un byte y abre los datos en modo binario, por lo que no interpreta el contenido según su extensión.

En lo sucesivo, n denota el número de bytes del archivo, $k \leq 256$ el número de valores distintos presentes, f_i la frecuencia del símbolo i y ℓ_i la longitud de su código. El flujo codificado requiere

$$B = \sum_{i=1}^{k} f_i \ell_i \qquad \text{y} \qquad \overline{\ell} = \frac{B}{n} \quad (n > 0). \tag{5}$$

Estos valores describen los bits de datos y no incluyen los metadatos del formato.

Cada codificación produce dos archivos que deben conservarse juntos: el flujo binario y la tabla de frecuencias. La decodificación requiere ambos y escribe un archivo nuevo. El nombre de salida se obtiene a partir de la tabla, no del nombre original completo.

4.2. Metodología y entorno de pruebas

4.2.1. Entorno experimental

Las mediciones se realizaron en un solo equipo, para que los resultados fueran comparables entre sí: MacBook Pro (identificador Mac15,3) con chip Apple M3 de ocho núcleos —cuatro de rendimiento y cuatro de eficiencia—, 8 GB de memoria y macOS Sequoia 15.1.1. Durante las pruebas se minimizaron los procesos en segundo plano, el equipo permaneció conectado a la corriente eléctrica y se utilizó `caffeinate` para evitar la suspensión mientras se ejecutaba el script. Se procesaron 21 archivos de distintos formatos, con tamaños desde 28 bytes hasta 2.2 GB, y se registró una medición por archivo.

Las mediciones de tamaño y tiempo se tomaron con la versión 1.0 de los programas. La sección 6 explica en qué difiere su formato del que se describe en este documento.

4.2.2. Pruebas funcionales

Para verificar la versión final, se compilaron ambos programas por separado con las opciones `-std=gnu11 -Wall -Wextra`, se construyeron entradas pequeñas y deterministas que cubren casos límite —archivo vacío, símbolo único, alfabeto completo y variantes de rutas— y se comparó cada archivo recuperado con su original mediante `cmp`. Estas pruebas verifican la recuperación exacta, pero no sustituyen un estudio de rendimiento con repeticiones y dispersión estadística.

4.3. Estructuras de datos

Tabla 6: Estructuras principales de la implementación.

Estructura	**Responsabilidad**
`Nodo`	Relaciona un byte, almacenado en `char`, con una frecuencia de tipo `long long`.
`Arbol`	Guarda frecuencia, carácter y punteros a los hijos izquierdo y derecho. Una hoja representa un símbolo.
`CodigoHuffman`	Contiene un arreglo de bytes para los bits del código y su longitud; existe una entrada por valor de byte.
`BufferBits`	Acumula hasta ocho bits y los escribe en el archivo de salida.

4.4. Implementación del codificador

1. `InicializarFrecuencias` establece las 256 frecuencias en cero.
2. `LeerArchivo_Frecuencia` recorre el archivo con `fgetc`; su resultado entero permite distinguir todos los bytes del indicador `EOF`.
3. `EscribirTablaFrecuencias` almacena extensión y frecuencias.
4. La función `ConstruirArbolHuffman` crea las hojas. Ordena los punteros con `QuickSortArboles` y fusiona los dos menores hasta obtener una raíz.
5. `GenerarCodigosHuffman` recorre las ramas y llena la tabla de códigos, usando cero a la izquierda y uno a la derecha.
6. `CodificarArchivo` vuelve a leer la entrada y escribe los códigos mediante `EscribirBits` y `EscribirBit`.
7. `ManejarUltimoByte` completa con ceros el último byte cuando quedan bits pendientes. Se liberan árbol, códigos y arreglos auxiliares.

El archivo se recorre dos veces: una para contar frecuencias y otra para codificar. No se carga completo en memoria. El siguiente fragmento corresponde al núcleo de construcción del árbol:

```
while(numArboles>1){
    QuickSortArboles(arboles, 0, numArboles-1);
    Arbol *arbolFusionado=FusionarArboles(arboles[0], arboles[1]);
    arboles[0]=arbolFusionado;
    for(int i=1; i<numArboles-1; i++){
        arboles[i]=arboles[i+1];
    }
    numArboles--;
}
```

Código 1: Fusión voraz en la implementación.

Las hojas se crean en orden creciente de valor de byte. QuickSort, con el último elemento como pivote, no es un ordenamiento estable: ante pesos iguales, el orden resultante depende de la posición previa de los árboles. Esto explica las decisiones de desempate del ejemplo de la sección 3.7. Como el procedimiento es determinista, el decodificador obtiene el mismo árbol si parte de las mismas hojas en el mismo orden, condición que garantiza la tabla de frecuencias.

4.5. Formato de almacenamiento

El archivo `<base>_codificacion.dat` comienza con `sizeof(long long)` bytes que representan el tamaño original en la representación nativa del equipo. Después aparecen $\lceil B/8 \rceil$ bytes con los códigos, del bit más significativo al menos significativo. No se almacena el árbol dentro de ese archivo.

La tabla `<base>_frecuencias.txt` contiene el encabezado literal `C F ASCCI`, la extensión en la segunda línea y una fila por símbolo presente. Los bytes imprimibles aparecen entre comillas simples y los demás como `0xNN`; cada fila incluye frecuencia y valor decimal. El encabezado es una etiqueta del formato: el alfabeto es de 256 bytes y no se limita a los 128 valores de ASCII.

Para la entrada de ocho bytes `AAAAABBC`, sin salto de línea, la tabla es:

```
C F ASCCI
txt
'A' 5 65
'B' 2 66
'C' 1 67
```

Código 2: Tabla de frecuencias de un ejemplo mínimo.

La implementación asigna `00` a C, `01` a B y `1` a A. La secuencia resultante es `11111010100`, de 11 bits, y se completa a dos bytes con cinco ceros. Con un `long long` de ocho bytes, como en el equipo de pruebas, la cabecera ocupa ocho bytes, el archivo binario diez y la tabla 41. El total de 51 bytes supera los ocho originales: el ahorro de bits del mensaje no compensa los metadatos en esta entrada pequeña.

4.6. Implementación del decodificador

`LeerArchivo` recupera extensión y frecuencias; luego se reconstruye el árbol con el mismo procedimiento de ordenamiento y fusión. Reproducir el orden de las hojas y las decisiones ante empates es necesario, porque la tabla no contiene códigos explícitos.

`DecodificarArchivo` lee el tamaño original y recorre los bits del flujo. Cada llegada a una hoja produce un byte y reinicia el recorrido en la raíz. El contador de bytes escritos limita la salida al tamaño declarado, por lo que los ceros de relleno no se procesan como símbolos adicionales, el problema descrito en la sección 3.8.

Se consideran dos casos particulares: para una entrada vacía, el árbol es nulo y el tamaño es cero; para un único símbolo, el decodificador repite el byte de la única hoja tantas veces como indica la cabecera. El codificador asigna un código de un bit a este

último caso.

4.7. Ejecución y automatización

Desde la carpeta que contiene los archivos fuente:

```
gcc -std=gnu11 -Wall -Wextra Codificacion_Huffman.c tiempo.c -o
    Codificacion_Huffman
gcc -std=gnu11 -Wall -Wextra Decodificacion_Huffman.c tiempo.c -o
    Decodificacion_Huffman
printf 'AAAAABBC' > ejemplo.txt
./Codificacion_Huffman ejemplo.txt
./Decodificacion_Huffman ejemplo_codificacion.dat ejemplo_frecuencias.txt
cmp ejemplo.txt ejemplo_frecuencias_decodificado.txt
```

Código 3: Compilación y recuperación verificable.

La biblioteca `tiempo.c` produce una advertencia por una variable `mega` sin uso, sin impedir la compilación. El comando `cmp` termina con código cero si el contenido coincide.

Para procesar varios archivos se crea, en la carpeta de los fuentes, la lista `archivos_a_codificar.txt`, con una ruta por línea, y se ejecuta el script desde esa misma carpeta:

```
bash Codificacion_Huffman.sh
```

Si el script se encuentra en otra carpeta, se indica su ruta; por ejemplo, `bash ../Scripts/Codificacion_Huffman.sh`. En macOS puede anteponerse `caffeinate` para evitar la suspensión durante ejecuciones largas. El script compila los programas, procesa cada entrada y compara el resultado con el original. Guarda dos registros, `codificacion.txt` y `decodificacion.txt`, que se reinician en cada ejecución. Las rutas con espacios se admiten sin comillas dentro de la lista.

4.8. Medición de tiempo y compresión

Se mide la operación que procesa los datos mediante llamadas a `uswtime`. En el codificador se mide la función `CodificarArchivo`; en el decodificador, la función `DecodificarArchivo`. El conteo de frecuencias y la construcción del árbol están fuera del intervalo. El tiempo real incluye las operaciones de entrada y salida realizadas dentro de la región medida. El tiempo de sistema mide CPU en modo núcleo; no equivale a toda la espera por dispositivos.

$$P_{\mathrm{CPU}} = 100\frac{t_{\mathrm{user}} + t_{\mathrm{sys}}}{t_{\mathrm{real}}}. \tag{6}$$

Los valores decimal y científico impresos corresponden a la misma medición. En intervalos demasiado breves, la resolución del reloj limita su utilidad.

Para $n > 0$, el codificador calcula el ahorro con ambos archivos:

$$A = 100\left(1 - \frac{S_{\text{dat}} + S_{\text{tabla}}}{n}\right), \qquad S_{\text{dat}} = \text{sizeof}(\texttt{long long}) + \left\lceil \frac{B}{8} \right\rceil. \tag{7}$$

Un ahorro negativo significa expansión. La entrada vacía no tiene un porcentaje definido bajo esta fórmula.

4.9. Análisis de complejidad

Se distinguen n bytes, $k \leq 256$ símbolos y B bits de datos. Para $k > 1$, la profundidad máxima del árbol es $k - 1$, así que $B \leq n(k - 1)$. Para $k = 1$, la convención de esta implementación produce $B = n$.

El conteo necesita $O(n)$ operaciones. El procesamiento del flujo cuesta $O(n + B)$: además de recorrer los datos, se escriben o consultan sus bits. El árbol tiene $2k - 1$ nodos para $k > 0$. El recorrido visita $O(k)$ nodos; copiar los códigos de las hojas añade un costo proporcional a sus longitudes.

En cada fusión se vuelve a ordenar el arreglo, cuyo tamaño disminuye de k a dos. QuickSort usa el último elemento como pivote y no garantiza particiones equilibradas. Sumando sus cotas de peor caso:

$$T_{\text{árbol}}(k) = O\left(\sum_{j=2}^{k} j^2\right) = O(k^3). \tag{8}$$

Bajo la hipótesis de particiones equilibradas, la suma sería $O(\sum_{j=2}^{k} j \log j) = O(k^2 \log k)$. Esta hipótesis no es una garantía para el pivote utilizado. No corresponde atribuir a esta construcción la cota $O(k \log k)$ de una implementación con cola de prioridad, descrita en el estado del arte.

Una cota superior para los programas es $O(n + B + k^3)$, omitiendo el costo menor de manejar nombres de archivo. Como el alfabeto de bytes es fijo, $k \leq 256$ y las longitudes de código están acotadas por 255, la dependencia respecto a n es $O(n)$. La memoria auxiliar también está acotada respecto a n: árbol, frecuencias, búfer y tabla de 256 códigos de 32 bytes cada uno. Este análisis supone operaciones de costo constante sobre los enteros nativos empleados y entradas representables en ellos.

5. Gráficos y diagramas esquemáticos

5.1. Flujo de los programas

La figura 3 muestra las dos salidas necesarias y su participación en la recuperación.

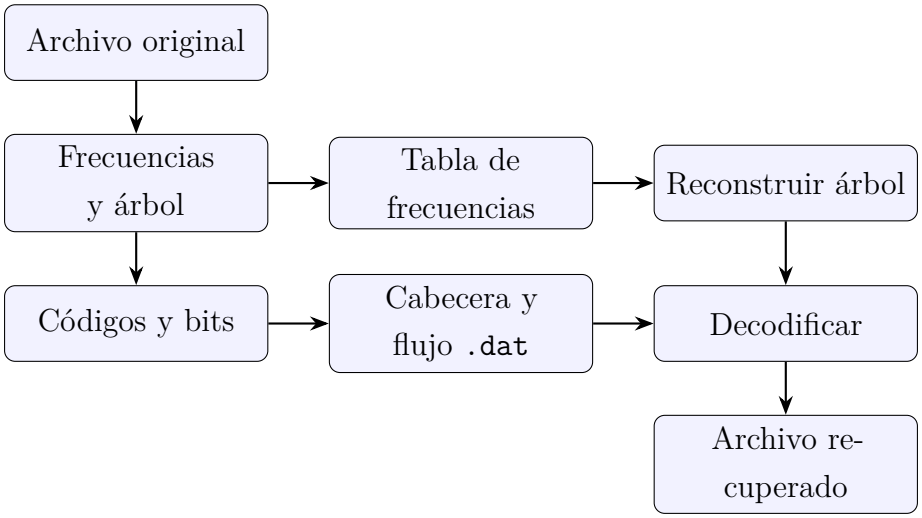


Figura 3: Codificación y decodificación con tabla externa.

5.2. Árbol del ejemplo

Para `AAAAABBC` se fusionan primero C y B, con peso conjunto tres; luego ese árbol se une a A, de peso cinco. La figura 4 representa el árbol obtenido y permite comprobar los 11 bits del mensaje.

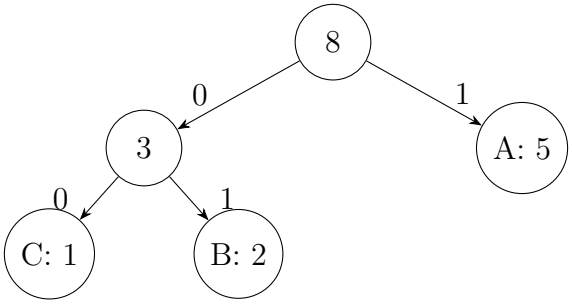


Figura 4: Códigos del ejemplo: $C = 00$, $B = 01$ y $A = 1$.

6. Resultados

6.1. Compresión obtenida

La tabla 7 resume las 21 pruebas; los nombres largos se abrevian para facilitar la lectura. Las mediciones se tomaron con la versión 1.0 de los programas, cuyo archivo codificado difiere del formato de la sección 4.5: comenzaba con un entero de cuatro bytes con el tamaño de una tabla de códigos; seguían el número de símbolos (cuatro bytes) y, para cada símbolo, su valor (un byte), la longitud de su código (cuatro bytes) y los bits del código; al final estaba el flujo de bits. El decodificador no utilizaba esa tabla, pues reconstruía el árbol a partir de la tabla de frecuencias, igual que la versión final.

La columna *Codificado* corresponde a ese archivo. La tabla de frecuencias en texto se generaba por separado y no se incluye en esa columna ni en el ahorro, calculado como $100(1 - S_{\text{cod}}/n)$.

Tabla 7: Tamaños y ahorro medidos con la versión 1.0 de los programas.

ID	Archivo abreviado	Original (B)	Codificado (B)	Ahorro (%)
1	RAR pequeño	530	1850	-249.06
2	abc.txt	28	193	-589.29
3	Lista ZIP	736	1854	-151.90
4	codificacion1.zip	532	1776	-233.83
5	Lista TXT	242	464	-90.95
6	Calaverita.png	429663	431115	-0.34
7	Mybrain.txt	2894	1871	35.35
8	Fondo JPEG	242941	242898	0.02
9	Arquitectura PDF	191932	192975	-0.54
10	Programación PDF	921058	912053	0.98
11	Ejercicios03.7z	17160935	17162479	-0.01
12	Sunshine MP3	5433506	5273617	2.94
13	PRACTICA.zip	32374692	32332072	0.13
14	Life PDF	98439043	95768206	2.71
15	Análisis PPTX	20156202	20157746	-0.01
16	Circuito.bmp	2507322	771420	69.23
17	Vid1.MOV	2205190700	2205192244	0.00
18	Vid2.MOV	1505923176	1505924720	0.00
19	Texto aleatorio de 1 GiB	1073741824	800975225	25.40
20	x64p.rar	1420641641	1420643185	0.00
21	x64o.rar	1749295439	1749296983	0.00

Los archivos pequeños presentan una expansión importante; por ejemplo, `abc.txt` pasa de 28 a 193 bytes, porque la tabla de códigos ocupa más que el propio mensaje. En la fila 5, el porcentaje registrado ($-90.95\,\%$) no corresponde a los tamaños, que dan $-91.74\,\%$; alguno de los dos valores se registró incorrectamente.

El resultado más favorable es `Circuito.bmp`, con 69.23 % de ahorro: un mapa de bits sin compresión previa y con pocos colores tiene una distribución de bytes muy desigual. `Mybrain.txt`, un texto en lenguaje natural, alcanza 35.35 %. El texto aleatorio de 1 GiB registra 25.40 %, valor que coincide con el esperado para 62 símbolos equiprobables, como letras mayúsculas, minúsculas y dígitos: el código óptimo asigna cinco bits a dos símbolos y seis a los 60 restantes, con longitud media $370/62 \approx 5.968$ bits; el ahorro resulta $1 - 5.968/8 \approx 25.40\,\%$, muy cerca del límite $1 - \log_2 62/8 \approx 25.57\,\%$ que impone la entropía.

En seis archivos ya comprimidos —`Ejercicios03.7z`, la presentación PPTX, los dos videos MOV y los dos archivos RAR grandes— el tamaño aumenta exactamente 1544 bytes. Esa cifra coincide con la tabla de códigos para los 256 valores de byte con códigos de ocho bits, $4 + 4 + 256(1 + 4 + 1) = 1544$ bytes; en ese caso el flujo ocupa lo mismo que la entrada y todo el aumento corresponde a la tabla. El resultado es consistente con una entropía cercana a ocho bits por byte (sección 3.5). Los ahorros de 0.00 % de los videos y de los RAR grandes están redondeados: sus tamaños codificados son ligeramente mayores que los originales.

Estos resultados muestran que el tamaño original no determina por sí solo la reducción. La distribución de frecuencias y el costo de los metadatos explican por qué un archivo grande puede no ahorrar espacio y un texto mucho menor sí hacerlo.

6.2. Tiempos de ejecución y ajuste

La tabla 8 presenta los tiempos de codificación y decodificación, junto con el tamaño del archivo recuperado.

Tabla 8: Tiempos medidos y tamaño recuperado; ID según la tabla anterior.

ID	Codificación (s)	Decodificación (s)	Recuperado (B)
1	0.0012218952	0.0012040138	530
2	0.0004141331	0.0003719330	28
3	0.0003609657	0.0003139973	739
4	0.0004961491	0.0004348755	532
5	0.0003128052	0.0003190041	242
6	0.0488059521	0.0340318680	429663

ID	Codificación (s)	Decodificación (s)	Recuperado (B)
7	0.0005140305	0.0005478859	2894
8	0.0259029865	0.0186607838	242941
9	0.0204520226	0.0150511265	191932
10	0.1067399979	0.0810709000	921058
11	1.7546558380	1.2377779484	17160935
12	0.5753290653	0.3993949890	5433506
13	3.4616010189	2.5062210560	32374692
14	10.3670279980	7.1660330296	98439043
15	2.1368360000	1.4801719189	20156202
16	0.0865399837	0.0725109577	2507323
17	236.4356610775	168.8322980404	2205190700
18	163.4552521706	115.5768148899	1505923176
19	93.6821660995	65.9858610630	1073741825
20	149.9963970184	102.4175059795	1420641641
21	180.9462549686	124.1385420000	1749295439

En tres pruebas el tamaño recuperado no coincide con el original: Lista ZIP recupera 739 bytes frente a 736; `Circuito.bmp`, 2507323 frente a 2507322, y el texto aleatorio, 1073741825 frente a 1073741824. La causa está en el decodificador de la versión 1.0, que procesaba el flujo hasta el fin del archivo: cuando los bits de relleno del último byte formaban el código de algún símbolo, se emitían símbolos adicionales. Es la situación descrita con `ABRACADABRA` en la sección 3.8, mensaje para el cual esa versión recupera `ABRACADABRAA`. La versión final almacena el tamaño original en la cabecera y se detiene al alcanzarlo (sección 4.6). En las demás pruebas los tamaños coinciden; aun así, la igualdad de tamaño no basta para demostrar la recuperación exacta, pues hace falta comparar el contenido.

Para visualizar la dependencia del tiempo respecto al tamaño se calcula un ajuste lineal por el origen sobre las 21 observaciones. Si $x_i = n_i/10^9$ y t_i es el tiempo medido, se minimiza $\sum_i (t_i - a x_i)^2$:

$$a = \frac{\sum_i x_i t_i}{\sum_i x_i^2}, \qquad \widehat{t}(n) = a \frac{n}{10^9}. \tag{9}$$

Se obtienen $a_c = 104.607943$ s por 10^9 bytes para codificación y $a_d = 73.332743$ s por 10^9 bytes para decodificación, equivalentes a unos 9.6 y 13.6 MB/s. La raíz del error cuadrático medio, $\sqrt{\sum_i (t_i - \widehat{t}_i)^2/21}$, es de 4.481688 s y 3.519449 s, respectivamente. Estos valores se calculan a partir de la tabla; no son mediciones adicionales. La figura 5 muestra datos y ajustes.

El modelo es descriptivo y no permite extrapolar garantías de rendimiento. Se registró

una medición por archivo, sin repeticiones ni intervalos de confianza, y cada tamaño corresponde a un contenido distinto. Los archivos mayores dominan el ajuste; en tamaños pequeños influyen la resolución temporal y los costos fijos. Una curva experimental, por sí sola, no demuestra una cota asintótica.

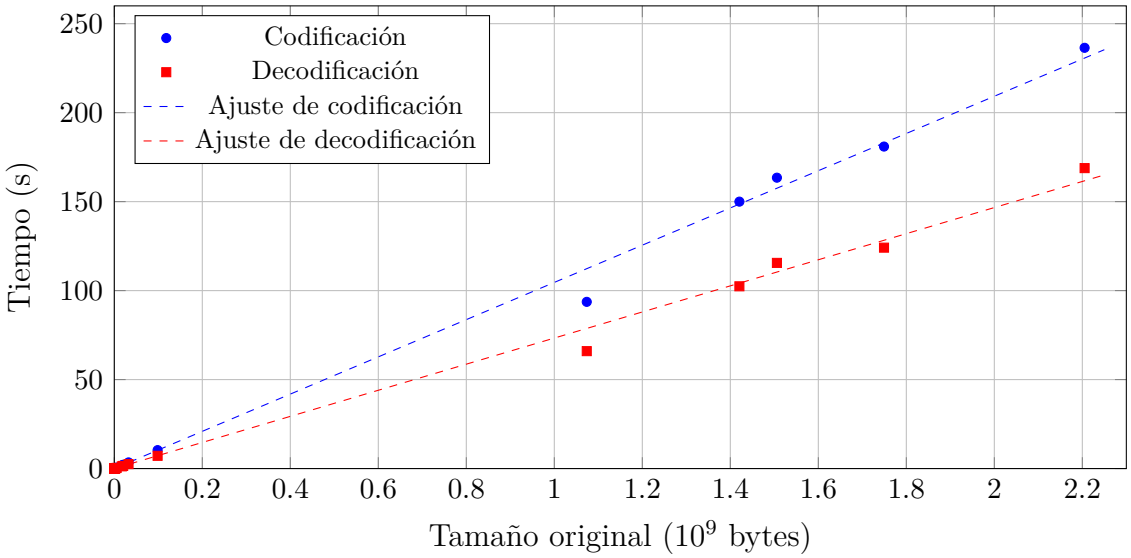


Figura 5: Tiempos medidos y ajustes lineales por el origen. Los puntos de menor tamaño quedan superpuestos cerca del origen; sus valores completos aparecen en la tabla 8.

6.3. Pruebas funcionales

La tabla 9 resume las pruebas funcionales de la versión final. Los tamaños están expresados en bytes; la cabecera de `long long` ocupa ocho bytes.

Tabla 9: Entradas verificadas mediante comparación íntegra del contenido.

Caso	**Original**	**.dat**	**Tabla**	**Igual**
`AAAAABBC`	8	10	41	Sí
`ABRACADABRA`	11	11	59	Sí
Vacío	0	8	14	Sí
4096 repeticiones de A	4096	520	26	Sí
256 bytes, repetidos 16 veces	4096	4104	2881	Sí
Texto UTF-8; ruta con espacios	17500	9258	267	Sí
Sin extensión	999	217	44	Sí
Archivo oculto	105	40	61	Sí
Directorio con puntos	260	41	33	Sí

Para reproducir las entradas, el caso de símbolo único contiene 4096 bytes `A`; el alfabeto completo contiene los valores 0 a 255 repetidos 16 veces; el texto UTF-8 repite 500 veces la

línea “Codificación de Huffman en ESCOM.” seguida de un salto de línea. Los tres últimos casos contienen respectivamente `ABC` repetido 333 veces, `01234` repetido 21 veces y `ABBA` repetido 65 veces. Los dos primeros casos usan extensión `txt`; vacío, único y alfabeto completo usan `bin`; el texto UTF-8 usa `txt`; los tres restantes carecen de extensión. Esta información permite reproducir también las tablas auxiliares. Para `ABRACADABRA`, el archivo `.dat` contiene la cabecera con el valor 11 y los tres bytes `69 EE 68` calculados en la sección 3.8.

6.4. Limitaciones

El formato emplea un entero nativo, sin definir orden de bytes ni versión, por lo que no garantiza interoperabilidad entre arquitecturas. La extensión se recupera en un búfer de 64 bytes. Los archivos de salida se sobrescriben y dos entradas con el mismo nombre base pueden compartir sus nombres de archivos auxiliares.

No existe checksum ni validación exhaustiva de datos truncados o tablas alteradas. En particular, alcanzar el fin del flujo antes del tamaño declarado no siempre produce un diagnóstico de error. Las pruebas anteriores verifican pares válidos producidos por el codificador; no demuestran robustez frente a entradas arbitrarias. La comparación del resultado sigue siendo necesaria.

Las mediciones de tiempo y tamaño no se repitieron con la versión final. Su recorrido de los datos y la región medida son comparables a los de la versión 1.0, pero los tamaños codificados de la tabla 7 no corresponden al formato final.

7. Cuestionario

1. **¿La codificación ofrece una ventaja de tamaño en promedio?** No en todos los casos. Las pruebas incluyen ahorros altos, como 69.23 % en `Circuito.bmp`, resultados casi nulos en archivos ya comprimidos y expansiones en archivos pequeños. El promedio simple de los 21 porcentajes es cercano a $-56\,\%$, dominado por los archivos de pocos bytes, mientras que el ahorro ponderado por tamaño, $1 - \sum S_{\text{cod}} / \sum n$, es de 3.41 %. La respuesta depende de la colección de archivos, de cómo se calcule el promedio y de que se incluya el espacio auxiliar.

2. **¿Los tiempos de codificación o decodificación son muy grandes?** Depende del uso. El archivo de aproximadamente 2.21 GB requirió 236.44 s para codificar y 168.83 s para decodificar, unos 9.3 y 13.1 MB/s. Es un costo considerable para una operación interactiva, aunque puede ser aceptable en procesamiento por lotes. No debe

generalizarse a otros equipos.

3. **¿Qué proporción temporal guardan ambos procesos?** Para `Vid1.MOV`, el cociente entre codificación y decodificación es aproximadamente 1.40; para `x64o.rar`, 1.46, y el cociente de las pendientes ajustadas es 1.43. No es constante. Los tiempos no incluyen las etapas previas a la región cronometrada, por lo que la diferencia no se atribuye al conteo inicial ni a la construcción del árbol.

4. **¿Ocurrieron pérdidas de información?** El algoritmo es sin pérdida, pero en tres pruebas de la versión 1.0 la recuperación no fue exacta: el archivo recuperado tuvo uno o tres bytes adicionales, producidos al decodificar el relleno final. La versión final corrige el problema mediante la cabecera con el tamaño original, y sus nueve pruebas funcionales, comparadas con `cmp`, recuperaron exactamente la entrada. El error provino del formato, no del código de Huffman.

5. **¿El comportamiento experimental era el esperado?** En lo general, sí. La variación del ahorro es compatible con distribuciones de frecuencias distintas: los archivos ya comprimidos no se reducen, como predice su entropía cercana a ocho bits por byte, y el texto aleatorio alcanza el ahorro previsto para un alfabeto de 62 símbolos equiprobables. La dependencia aproximadamente lineal del tiempo es coherente con un alfabeto fijo de bytes, aunque las mediciones aisladas no constituyen una prueba formal de complejidad.

6. **¿Qué características favorecen la compresión de una imagen?** Una distribución desigual de bytes y un tamaño suficiente para amortizar los metadatos. Una imagen sin compresión previa, con pocos colores y grandes regiones uniformes, como `Circuito.bmp`, cumple estas condiciones. En cambio, PNG y JPEG ya incluyen una etapa de codificación de entropía —DEFLATE y Huffman, respectivamente [5, 14]—, por lo que `Calaverita.png` y el fondo JPEG prácticamente no cambian de tamaño. El programa no interpreta píxeles ni detecta regiones de color.

7. **¿Qué características favorecen la compresión de texto?** Que ciertos bytes aparezcan con mayor frecuencia y que el archivo sea lo bastante grande para compensar la tabla. El texto en lenguaje natural usa un subconjunto reducido de los 256 valores con frecuencias desiguales, como muestra `Mybrain.txt`. La implementación codifica bytes individuales con un modelo de orden cero; no reconoce palabras ni sustituye secuencias repetidas, tarea propia de los métodos de diccionario.

8. **¿Cuáles son tres aplicaciones posibles?** La compresión de archivos con DEFLATE (ZIP, gzip y PNG), donde Huffman codifica los símbolos que produce LZ77 [5]; la codificación de entropía de imágenes y audio en JPEG y MP3 [14, 21], y la compresión

de cabeceras en HTTP/2 con un código estático [22]. En cada caso debe evaluarse el costo de compartir el modelo y el tiempo de procesamiento.

9. **¿Existió un entorno controlado?** Parcialmente. Se utilizó un único MacBook Pro M3, con alimentación continua, prevención de suspensión mediante `caffeinate` y reducción de procesos ajenos. Sin embargo, se registró una sola medición por archivo y no se cuantificó la variabilidad, de modo que el control descrito no elimina toda la incertidumbre de las mediciones.

10. **¿Qué recomendaciones se desprenden de la práctica?** Verificar el contenido con `cmp`; conservar juntos el flujo y la tabla; probar entradas vacías, símbolos únicos y todos los valores del byte; contabilizar los metadatos; almacenar el tamaño original o un símbolo de fin, y repetir las mediciones bajo condiciones comparables. Como mejoras algorítmicas convendría evaluar una cola de prioridad en lugar de los ordenamientos completos y un código canónico en lugar de la tabla de frecuencias en texto.

8. Conclusiones

Al estudiar el algoritmo comprendí por qué la decisión voraz de fusionar los dos pesos mínimos produce un código óptimo: esa elección no compromete la solución, porque siempre existe un árbol óptimo con esos dos símbolos como hermanos, y el problema restante conserva la misma estructura. El ejemplo construido paso a paso me mostró, además, que los empates admiten varios árboles igualmente óptimos, lo que obliga al decodificador a repetir exactamente las decisiones del codificador.

Al implementar el método identifiqué que el árbol es sólo una parte del sistema: para recuperar un archivo también debo conservar el modelo de frecuencias, el tamaño original y el orden de lectura de los bits. La cabecera permite distinguir los datos del relleno final —las tres recuperaciones con bytes de más se explican precisamente por ese relleno— y los casos vacío y de símbolo único requieren tratamiento explícito.

Los resultados me permiten distinguir el ahorro del mensaje del ahorro del formato completo. Un código puede reducir los bits de los símbolos y aun así producir un archivo mayor al incorporar sus metadatos, y ningún código por símbolos reduce un archivo cuya entropía ya es cercana a ocho bits por byte. También comprobé que la exactitud debe verificarse por contenido; un algoritmo teóricamente sin pérdida no basta para validar una implementación.

Finalmente, separé el tamaño del archivo del tamaño del alfabeto en el análisis de complejidad. Para esta implementación, repetir QuickSort no equivale a usar una cola

de prioridad, mientras que el alfabeto fijo permite describir el procesamiento respecto al tamaño de entrada como lineal. Esta distinción, junto con las variantes revisadas en el estado del arte —códigos canónicos, de longitud limitada y adaptativos—, conecta las decisiones de programación con el análisis formal de algoritmos.

Referencias bibliográficas

[1] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, núm. 9, pp. 1098–1101, septiembre de 1952. doi: 10.1109/JRPROC.1952.273898.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.ª ed. Cambridge, MA, EE. UU.: MIT Press, 2009, cap. 16, sec. 16.3.

[3] A. Moffat, "Huffman Coding," *ACM Computing Surveys*, vol. 52, núm. 4, art. 85, pp. 1–35, 2019. doi: 10.1145/3342555.

[4] J. Ziv y A. Lempel, "A Universal Algorithm for Sequential Data Compression," *IEEE Transactions on Information Theory*, vol. 23, núm. 3, pp. 337–343, mayo de 1977.

[5] P. Deutsch, "DEFLATE Compressed Data Format Specification version 1.3," RFC 1951, Internet Engineering Task Force, mayo de 1996.

[6] C. E. Shannon, "A Mathematical Theory of Communication," *The Bell System Technical Journal*, vol. 27, núm. 3, pp. 379–423, julio de 1948.

[7] R. M. Fano, "The Transmission of Information," Technical Report núm. 65, Research Laboratory of Electronics, Massachusetts Institute of Technology, Cambridge, MA, EE. UU., marzo de 1949.

[8] L. G. Kraft, "A Device for Quantizing, Grouping, and Coding Amplitude-Modulated Pulses," tesis de maestría, Massachusetts Institute of Technology, Cambridge, MA, EE. UU., 1949.

[9] B. McMillan, "Two Inequalities Implied by Unique Decipherability," *IRE Transactions on Information Theory*, vol. 2, núm. 4, pp. 115–116, diciembre de 1956.

[10] T. M. Cover y J. A. Thomas, *Elements of Information Theory*, 2.ª ed. Hoboken, NJ, EE. UU.: Wiley-Interscience, 2006, cap. 5.

[11] K. Sayood, *Introduction to Data Compression*, 4.ª ed. Waltham, MA, EE. UU.: Morgan Kaufmann, 2012, cap. 3.

[12] J. van Leeuwen, "On the Construction of Huffman Trees," en *Proceedings of the 3rd International Colloquium on Automata, Languages and Programming (ICALP)*, Edimburgo, Reino Unido, 1976, pp. 382–410.

[13] E. S. Schwartz y B. Kallick, "Generating a Canonical Prefix Encoding," *Communications of the ACM*, vol. 7, núm. 3, pp. 166–169, 1964.

[14] ITU-T, "Information Technology – Digital Compression and Coding of Continuous-Tone Still Images – Requirements and Guidelines," Recomendación T.81, septiembre de 1992.

[15] L. L. Larmore y D. S. Hirschberg, "A Fast Algorithm for Optimal Length-Limited Huffman Codes," *Journal of the ACM*, vol. 37, núm. 3, pp. 464–473, 1990.

[16] R. G. Gallager, "Variations on a Theme by Huffman," *IEEE Transactions on Information Theory*, vol. 24, núm. 6, pp. 668–674, 1978.

[17] D. E. Knuth, "Dynamic Huffman Coding," *Journal of Algorithms*, vol. 6, núm. 2, pp. 163–180, 1985.

[18] J. S. Vitter, "Design and Analysis of Dynamic Huffman Codes," *Journal of the ACM*, vol. 34, núm. 4, pp. 825–845, 1987.

[19] I. H. Witten, R. M. Neal y J. G. Cleary, "Arithmetic Coding for Data Compression," *Communications of the ACM*, vol. 30, núm. 6, pp. 520–540, 1987.

[20] J. Duda, "Asymmetric Numeral Systems: Entropy Coding Combining Speed of Huffman Coding with Compression Rate of Arithmetic Coding," arXiv:1311.2540, 2013.

[21] ISO/IEC, "Information Technology – Coding of Moving Pictures and Associated Audio for Digital Storage Media at up to about 1,5 Mbit/s – Part 3: Audio," norma ISO/IEC 11172-3, 1993.

[22] R. Peon y H. Ruellan, "HPACK: Header Compression for HTTP/2," RFC 7541, Internet Engineering Task Force, mayo de 2015.

[23] J. Alakuijala y Z. Szabadka, "Brotli Compressed Data Format," RFC 7932, Internet Engineering Task Force, julio de 2016.

[24] Y. Collet y M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, Internet Engineering Task Force, febrero de 2021.